

Brain Core Developer Production Readiness Pack

Review target: `review-brain-core-codebase` at `386e8da`

Baseline merged main: `origin/main` at `f75c467`

Audience: Brain engineering and deployment owners

Last updated: 2026-06-21

This pack is the developer-facing source of truth for the current technical state, the production-readiness gap, and the diagrams the team should maintain as code changes.

Assessment Of The Attached Docs

The attached documents are a strong starting set for engineering alignment:

- `Brain-Technical-Architecture.pdf` gives a useful five-page overview of the six-layer model, money path, process topology, security model, and deployment target.
- The standalone architecture, money-path, MCP, deployment, and process-role diagrams are readable and mostly match the code shape.
- `Brain-Core-Production-Readiness.docx` has the right structure for a dev-team status memo: executive snapshot, architecture brief, hardening state, production path, risk register, CI posture, and open checklist.

They should not be treated as final production docs without updates:

- The Word summary contains literal `[object Object]` in status cells. That is a generation bug and makes the status tables unsafe to send.
- The docs are anchored to `main @ f75c467` on 2026-06-20. The current working branch is `386e8da` and includes reservation hardening plus profile evidence gates.
- The readiness status changed from "staging yellow" to "staging red" under the stricter branch logic because required evidence is still scaffolded: DB-role evidence is not configured in this environment and Base Sepolia E2E is not exercised.
- The money-path diagram does not show the new authoritative reservation handoff: lock source account, lock latest balance snapshot, recheck available balance net of active reservations, reserve, transition, enqueue.
- The process/DB-role diagram is accurate, but the dev-team version should call out the deferred hardening item: tenant deletion should move behind a control-plane path so the broad deletion role is not part of the public API process.
- The readiness docs should explicitly point to `pnpm run readiness:evidence -- --profile staging` as the release-candidate diligence artifact.

Current Readiness

Local command results on this branch:

```
node scripts/production-readiness.mjs --json --profile=demo
node scripts/production-readiness.mjs --json --profile=staging
node scripts/production-readiness.mjs --json --profile=mainnet
```

Profile	Status	Why
demo	green	Code, guards, seeded demo path, and demo-scope evidence pass.
staging	red	DB isolation evidence is still scaffolded/configuration-only locally, and Base Sepolia on-chain executor E2E is not exercised.
mainnet	red	External BrainEscrow audit is pending, money-path rail evidence is not exercised, live rail evidence is not exercised, and mainnet operational proof is absent.

The key shift is that status and evidence are separate. A row can be configured or scaffolded and still fail a staging/mainnet profile if that profile requires exercised evidence.

Architecture Summary

Brain Core is a policy-gated financial control plane for autonomous agents. Agents can read financial memory and propose actions, but every money movement must pass the deterministic pre-execution gate before dispatch.

The working architecture has these hard boundaries:

- Raw owns immutable source evidence.
- Canonical owns rich normalized domain records and projection state.
- Ledger owns machine-readable financial truth and PaymentIntent rows.
- Wiki owns human-readable memory and narrative Q&A.
- Policy reads Ledger, never Wiki, and emits deterministic decisions.
- Execution owns PaymentIntent orchestration, approval, gate execution, outbox handoff, and rail settlement.
- MCP exposes read and propose surfaces only. It does not expose execute.
- Audit owns the append-only hash chain, Merkle publication, and proof surface.

Agent and on-chain event handling needs its own operational map. The current repo has good component docs, but they are spread across protocol, MCP, ADR, and smart-contract folders. Use docs/diligence/agent-and-onchain-event-map.md as the dev-team bridge between agent actions, audit/domain events, on-chain logs, event watchers, and reconciliation.

Money Path Invariant

For ledger-account-backed payments, the live handoff must be:

1. Load approved `PaymentIntent`.
2. Run the deterministic pre-execution gate as a preflight.
3. Open one tenant-scoped transaction.
4. Transition `approved` $\rightarrow$ `dispatching`.
5. Lock the source account row.
6. Lock the latest balance snapshot when present.
7. Recheck `available_balance - active_reservations >= amount`.
8. Insert an active reservation.
9. Insert the execution outbox row with `reservation_id`.
10. Commit.

The outbox worker then claims, dispatches, validates the typed receipt, and settles:

- Success: `dispatching` $\rightarrow$ `executed`, append execution receipt, consume reservation, record agent spend.
- Deterministic no-money-moved failure: `dispatching` $\rightarrow$ `failed`, release reservation.
- Ambiguous outcome: keep the `PaymentIntent` `dispatching` and route the outbox row to retry/reconciliation.

See `docs/diligence/diagrams/money-path-locked-reservation.mmd`.

Release-Candidate Gates

For a staging release candidate, do not rely on "configured" or "scaffolded" as proof for core safety rows.

Required staging evidence:

- `production-readiness --profile staging` exits green.
- `readiness:evidence -- --profile staging` is attached to the release record.
- DB-role evidence is at least configured and preferably exercised against the staging Postgres role model.
- Base Sepolia on-chain executor E2E is exercised with provisioned fixtures.
- Money-movement E2E runs against staging API, Postgres, Redis, and rail sandbox/testnet endpoints.
- Worker lease, projection lag, outbox age, audit verifier, and quarantine metrics are visible.

Required mainnet evidence:

- External BrainEscrow audit is complete and `contracts/audit-status.json` is approved from the auditor report.
- `verify-audited-build` matches source tree, compiler, creation bytecode, and immutable-masked runtime bytecode.
- Mainnet escrow boot fence has audit receipt/attestation and deployed bytecode verification.
- All money-path rails required for the deployment have exercised evidence.
- Rollback and incident drills are recorded.

See `docs/diligence/diagrams/readiness-evidence-gates.mmd`.

Agent And On-Chain Event Documentation

The dev team should maintain a single event catalog and reconciliation runbook for agent and on-chain behavior. Required source docs:

- docs/diligence/agent-and-onchain-event-map.md
- docs/diligence/diagrams/agent-onchain-event-map.mmd
- docs/diligence/diagrams/onchain-event-reconciliation-lifecycle.mmd

This matters for production because agent autonomy and on-chain execution are only credible when engineers can trace:

- agent action -> off-chain audit/domain event,
- proposal -> PaymentIntent -> gate -> outbox -> receipt,
- contract log -> durable watcher cursor -> reconciliation row,
- incident -> revoke/pause/quarantine action -> audit proof.

Do not claim production-ready autonomy unless the event watcher, replay, dead-letter, reorg, dashboard, and incident-response paths are documented and exercised for the target profile.

Production Workstreams

Workstream	Owner	Exit criteria
External contract audit	Security/Protocol	Auditor engaged, report delivered, zero open critical/high findings, <code>audit-status.json</code> approved, audited build verified.
Azure staging deploy	Platform	Terraform applied, role URLs wired through Key Vault, API/worker split deployed, staging E2E green.
Base Sepolia fixtures	Protocol/Platform	RPC, deployed smart account, funded throwaway key, granted session key, target contract/data, CI vars/secrets set.
ACH/Plaid settlement proof	Integrations	Plaid sandbox transfer and webhook settlement verified end to end, or ACH remains explicitly dispatch/reconcile-only.
Runtime dashboards	Platform/Ops	Dashboards for worker leases, projection lag, outbox age, reservations age, audit verifier health, dead letters, and rail failure rates.
Tenant deletion control-plane	Platform/Security	Broad tenant-deletion role removed from public API runtime; deletion is routed through operator/control-plane workflow.
Partner connector runtime	Integrations/Security	Out-of-process partner runtime with signed manifests, resource limits, network restrictions, revocation, and low-trust provenance until corroborated.

Code Anchors

- Gate: `shared/src/gate/gate.ts`
- PaymentIntent execution: `services/execution/src/payment-intents/PaymentIntentService.ts`
- Reservation writer: `services/ledger/src/repository/reservations.ts`

- Outbox service: `services/execution/src/outbox/OutboxService.ts`
- Outbox worker: `services/execution/src/outbox/worker.ts`
- Readiness aggregator: `scripts/production-readiness.mjs`
- Evidence report: `scripts/readiness-evidence.mjs`
- DB roles: `infra/db-roles.sql`
- Runtime role assertions: `services/api/src/composition/runtime-db-roles.ts`
- Process topology: `services/api/src/composition/process-roles.ts`
- Partner connector isolation: `services/raw/src/adapters/isolation.ts`
- Audit approval source: `contracts/audit-status.json`

Documentation Maintenance Rules

- Keep generated PDFs/DOCX in sync with these markdown and Mermaid sources.
- Never publish a status table that renders `[object Object]` or any non-text badge placeholder.
- Every production-readiness claim should name one of:
 - code anchor,
 - CI guard,
 - runtime boot fence,
 - staging/mainnet evidence artifact,
 - risk register entry.
- Update `resources/changelog.md`, `architecture/readiness-summary.md`, and `architecture/enterprise-readiness.md` when readiness semantics change.
- Run:

```
pnpm run check-docs-drift
pnpm run check-no-em-dashes
node scripts/production-readiness.mjs --json --profile=staging
pnpm run readiness:evidence -- --profile staging
```

Verdict

The attached docs are good conceptual drafts, but they are not yet sufficient as the developer production-readiness packet. They need the current branch state, the locked reservation handoff, profile evidence gates, and corrected status tables. This pack and the Mermaid sources below fill those gaps.